

# AegisVault-J: Secure Encrypted Container System

*Project for Product Development - II report submitted to the Biju Patnaik University of Technology*

*for the partial fulfillment for the award of the degree of*

**Bachelor of Technology**

*in*

**Computer Science and Engineering**

*by*

**Ayshi Shannidhya  
Panda**

*Regd. No.:*

2301341017

*SIC: 23BCSN22*

**Ashish Singh**

*Regd. No.:*

2301341014

*SIC: 23BCSN20*

**Avee Ranjan**

*Regd. No.:*

2301341016

*SIC: 23BCSN26*

**Ashish Kumar**

*Regd. No.:*

2301341013

*SIC: 23BCSN03*

*under the supervision of*

**Mrs. Snehanjali Pradhan**

Assistant Professor, Department of Computer Science and Engineering

Silicon Institute of Technology, Sambalpur, Odisha



**Silicon**  
*...beyond teaching*

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**SILICON INSTITUTE OF TECHNOLOGY**

**SAMBALPUR-768200, ODISHA, INDIA**

**APRIL 2026**



Department of Computer Science and Engineering  
**Silicon Institute of Technology**  
Sambalpur-768200, Odisha, India

27 April 2026

## **CERTIFICATE**

This is to certify that the work contained in the Project for Product Development - II entitled “**AegisVault-J: Secure Encrypted Container System**”, submitted by **Ayshy Shan-nidhya Panda (Regd. No.: 2301341017)**, **Ashish Singh (Regd. No.: 2301341014)**, **Avee Ranjan (Regd. No.: 2301341016)**, and **Ashish Kumar (Regd. No.: 2301341013)** for the award of the degree of **Bachelor of Technology** to the **Biju Patnaik University of Technology, Rourkela, Odisha** is a record of bonafide research works carried out by them under my direct supervision and guidance.

I considered that the Project for Product Development - II has reached the standards and fulfilling the requirements of the rules and regulations relating to the nature of the degree. The contents embodied in the Project for Product Development - II have not been submitted for the award of any other degree or diploma in this or any other university.

**Date:** 27 April 2026

**Place:** SIT, Sambalpur

**Mrs. Snehanjali Pradhan**

Supervisor

Dept. of CSE, SIT

**Dr. Bijaya Shankar**

**Tripathy**

Assistant Professor

HOD, Dept. of CSE, SIT

**External Examiner**

27 April 2026

## Declaration

We certify that

- (a) The work contained in the report is original and has been done by ourselves under the supervision of our supervisor.
- (b) The work has not been submitted to any other Institute for any degree or diploma.
- (c) We have conformed to the norms and guidelines given in the Ethical Code of Conduct of the University.
- (d) Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them by citing them in the text of the report and giving their details in the references.
- (e) Whenever we have quoted written materials from other sources and due credit is given to the sources by citing them.

**Date:** 27 April 2026

**Place:** SIT, Sambalpur

---

**Ayshi Shannidhya**

**Panda**

*Regd. No.:*

2301341017

---

**Ashish Singh**

*Regd. No.:*

2301341014

---

**Avee Ranjan**

*Regd. No.:*

2301341016

---

**Ashish Kumar**

*Regd. No.:*

2301341013

27 April 2026

## Acknowledgements

We would like to express our profound gratitude to the ones who have significantly contributed to the completion of this report. Foremost, we would like to express our sincere gratitude to our Supervisor, Mrs. Snehanjali Pradhan, Assistant Professor, Department of Computer Science and Engineering, SIT, Sambalpur for providing us intellectual guidance, sensible and timely advice, continuous support and motivation throughout our B. Tech. project work. Their invaluable comments, suggestions and motivations encouraged us from the beginning till the completion of our project work.

This report is prepared as part of the B.Tech Major Project work on secure storage systems. The project reflects practical learning in software engineering, cryptographic design, secure key management, enterprise deployment, cloud synchronization, and application-level file protection. The implementation emphasizes realistic boundaries and clear user awareness of what is protected and what is not.

We are fortunate enough to get valuable inputs anytime we needed, for which we are really thankful to our esteemed supervisor.

**Date:** 27 April 2026

**Place:** SIT, Sambalpur

---

**Ayshy Shannidhya**

**Panda**

*Regd. No.:*

*2301341017*

---

**Ashish Singh**

*Regd. No.:*

*2301341014*

---

**Avee Ranjan**

*Regd. No.:*

*2301341016*

---

**Ashish Kumar**

*Regd. No.:*

*2301341013*

## **Abstract**

AegisVault-J is a secure encrypted container application designed to store sensitive files inside a single portable vault file. The system is intended for users who need controlled, user-managed encryption without relying on cloud storage or full-disk encryption. It uses AES-256-GCM for authenticated encryption of vault contents and Argon2id for deriving a key from the user's password. The design follows a strict operational model: files are imported into the vault, encrypted inside the container, locked when not in use, and exported only when the user intentionally chooses to access them externally.

In this major project phase, the system has been significantly extended with four enterprise-grade features: hardware-backed key protection using OS-level keystores, encrypted cloud vault synchronization via local folders and WebDAV, detailed access audit logging with searchable viewer, and enterprise-level deployment support including cascading policy configuration, multi-vault management, and deployment reporting. These enhancements elevate AegisVault-J from a personal file vault to an enterprise-ready secure storage platform.

This report presents the problem context, system architecture, cryptographic design, security features, implementation stack, testing results, audit observations, limitations, and future enhancement opportunities. The analysis is intentionally conservative and avoids claims of absolute security.

**Keywords:** Encrypted container, AES-GCM, Argon2id, JavaFX, secure storage, virtual file system, authenticated encryption, hardware key protection, cloud synchronization, audit logging, enterprise deployment.

## **Acronyms**

|      |                                         |
|------|-----------------------------------------|
| AES  | Advanced Encryption Standard            |
| API  | Application Programming Interface       |
| ASIC | Application-Specific Integrated Circuit |
| CLI  | Command-Line Interface                  |
| CSE  | Computer Science and Engineering        |
| FUSE | Filesystem in Userspace                 |
| GCM  | Galois/Counter Mode                     |
| GPU  | Graphics Processing Unit                |
| GUI  | Graphical User Interface                |
| IV   | Initialization Vector                   |
| JIT  | Just-In-Time (Compilation)              |
| JVM  | Java Virtual Machine                    |
| KDF  | Key Derivation Function                 |
| KEK  | Key Encryption Key                      |
| MVK  | Master Vault Key                        |
| UI   | User Interface                          |
| VFS  | Virtual File System                     |

## Notations

|           |                                                 |
|-----------|-------------------------------------------------|
| $K_{MVK}$ | Master Vault Key (256-bit symmetric key)        |
| $K_{KEK}$ | Key Encryption Key (derived from user password) |
| $P$       | User password                                   |
| $S$       | Salt value for key derivation                   |
| $IV$      | Initialization Vector for AES-GCM               |
| $T$       | Authentication tag produced by GCM mode         |
| $C$       | Ciphertext output of encryption                 |
| $M$       | Memory cost parameter for Argon2id (64 MB)      |

## List of Figures

|     |                                                                                                                                                             |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Operational workflow of AegisVault-J. The vault enforces a deliberate sequence of actions rather than hidden background encryption. . . . .                 | 7  |
| 3.2 | Layered architecture of AegisVault-J. The design separates presentation, business logic, logical file organization, cryptography, and persistent storage. . | 8  |
| 3.3 | Module design of AegisVault-J showing the four core modules and their interactions. . . . .                                                                 | 10 |
| 4.1 | Password-to-vault access flow. Authentication succeeds through correct cryptographic decryption rather than direct password comparison. . . . .             | 14 |
| 5.1 | Security model of AegisVault-J showing the boundary between protected vault data and unprotected external data. . . . .                                     | 18 |
| 6.1 | Vault creation and login screen of AegisVault-J. Users can create a new vault or open an existing one with password authentication. . . . .                 | 22 |
| 6.2 | Main vault file browser of AegisVault-J. The interface provides a familiar file manager layout with vault-specific operations in the toolbar. . . . .       | 23 |
| 6.3 | Import/Export dialog of AegisVault-J. The interface shows file selection, encryption progress, and security warnings for export operations. . . . .         | 24 |

## **List of Tables**

|     |                                                               |    |
|-----|---------------------------------------------------------------|----|
| 5.1 | Security model summary of AegisVault-J . . . . .              | 20 |
| 6.1 | Comparison of AegisVault-J with common alternatives . . . . . | 27 |

## **Contents**

|                                      |             |
|--------------------------------------|-------------|
| <b>Certificate</b>                   | <b>ii</b>   |
| <b>Declaration</b>                   | <b>iii</b>  |
| <b>Acknowledgements</b>              | <b>iv</b>   |
| <b>Abstract</b>                      | <b>v</b>    |
| <b>Acronyms</b>                      | <b>vi</b>   |
| <b>Notations</b>                     | <b>vii</b>  |
| <b>List of Figures</b>               | <b>viii</b> |
| <b>List of Tables</b>                | <b>ix</b>   |
| <b>Contents</b>                      | <b>x</b>    |
| <b>1 Introduction</b>                | <b>1</b>    |
| 1.1 Background . . . . .             | 1           |
| 1.2 Purpose of the Project . . . . . | 1           |
| 1.3 Scope of the Work . . . . .      | 2           |
| 1.4 Project Objectives . . . . .     | 2           |
| 1.5 Problem Context . . . . .        | 3           |
| 1.6 Motivation . . . . .             | 3           |
| 1.7 Design Philosophy . . . . .      | 3           |

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>2 Literature and Conceptual Context</b>        | <b>4</b>  |
| 2.1 Introduction . . . . .                        | 4         |
| 2.2 Full-Disk Encryption . . . . .                | 4         |
| 2.3 Archive-Based Encryption . . . . .            | 4         |
| 2.4 Cloud Vaults . . . . .                        | 5         |
| 2.5 Application-Level Container Systems . . . . . | 5         |
| <b>3 System Model and Architecture</b>            | <b>6</b>  |
| 3.1 Introduction . . . . .                        | 6         |
| 3.2 Frozen Operational Workflow . . . . .         | 6         |
| 3.3 Security Consequences of the Model . . . . .  | 6         |
| 3.4 What the System Is . . . . .                  | 7         |
| 3.5 What the System Is Not . . . . .              | 7         |
| 3.6 Layered Architecture . . . . .                | 8         |
| 3.7 JavaFX UI Layer . . . . .                     | 8         |
| 3.8 Service Layer . . . . .                       | 9         |
| 3.9 Virtual File System Layer . . . . .           | 9         |
| 3.10 Encryption Engine Layer . . . . .            | 9         |
| 3.11 Container File Layer . . . . .               | 9         |
| 3.12 Module Design . . . . .                      | 9         |
| 3.12.1 Presentation Module . . . . .              | 10        |
| 3.12.2 Vault Management Module . . . . .          | 10        |
| 3.12.3 Crypto Module . . . . .                    | 11        |
| 3.12.4 Storage Module . . . . .                   | 11        |
| 3.13 Enhanced Modules (Major Project) . . . . .   | 11        |
| 3.13.1 Audit Module . . . . .                     | 11        |
| 3.13.2 Hardware Key Protection Module . . . . .   | 11        |
| 3.13.3 Cloud Synchronization Module . . . . .     | 12        |
| 3.13.4 Enterprise Deployment Module . . . . .     | 12        |
| <b>4 Cryptographic Design</b>                     | <b>13</b> |
| 4.1 Introduction . . . . .                        | 13        |

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| 4.2      | Core Key Concepts . . . . .                 | 13        |
| 4.3      | Key Derivation and Unlock Flow . . . . .    | 13        |
| 4.4      | AES-256-GCM . . . . .                       | 14        |
| 4.5      | Argon2id . . . . .                          | 15        |
| 4.6      | Security Properties . . . . .               | 15        |
| 4.7      | Entropy Sources . . . . .                   | 15        |
| 4.8      | Mouse Randomness Feature . . . . .          | 15        |
| 4.8.1    | Design Rationale . . . . .                  | 15        |
| 4.8.2    | Processing Model . . . . .                  | 16        |
| 4.8.3    | Security Interpretation . . . . .           | 16        |
| <b>5</b> | <b>Security Features and Threat Model</b>   | <b>17</b> |
| 5.1      | Introduction . . . . .                      | 17        |
| 5.2      | Authenticated Encryption . . . . .          | 17        |
| 5.3      | Auto-Lock . . . . .                         | 17        |
| 5.4      | Key Wiping . . . . .                        | 18        |
| 5.5      | Export Warnings . . . . .                   | 18        |
| 5.6      | Password Strength Checking . . . . .        | 19        |
| 5.7      | Hardware-Backed Key Protection . . . . .    | 19        |
| 5.8      | Access Audit Logging . . . . .              | 19        |
| 5.9      | Protected Assets . . . . .                  | 19        |
| 5.10     | Out of Scope . . . . .                      | 20        |
| 5.11     | Boundary of Responsibility . . . . .        | 20        |
| <b>6</b> | <b>Implementation, Testing, and Results</b> | <b>21</b> |
| 6.1      | Introduction . . . . .                      | 21        |
| 6.2      | Language and Runtime . . . . .              | 21        |
| 6.3      | User Interface . . . . .                    | 21        |
| 6.4      | Application UI Flow . . . . .               | 21        |
| 6.4.1    | Vault Creation and Login Screen . . . . .   | 22        |
| 6.4.2    | Main Vault File Browser . . . . .           | 22        |
| 6.4.3    | Import and Export Operations . . . . .      | 23        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 6.5      | Cryptographic Libraries . . . . .                            | 23        |
| 6.6      | Build and Packaging . . . . .                                | 24        |
| 6.7      | Vault Lifecycle . . . . .                                    | 24        |
| 6.8      | Import and Export Operations . . . . .                       | 25        |
| 6.9      | Folder Hierarchy Management . . . . .                        | 25        |
| 6.10     | Password Change . . . . .                                    | 25        |
| 6.11     | Backup Utility . . . . .                                     | 25        |
| 6.12     | Password Strength Checker . . . . .                          | 25        |
| 6.13     | Unit Testing Strategy . . . . .                              | 26        |
| 6.14     | Coverage Areas . . . . .                                     | 26        |
| 6.15     | Security Audit Summary . . . . .                             | 26        |
| 6.16     | Observed JVM Limitations . . . . .                           | 26        |
| 6.17     | Comparison with VeraCrypt . . . . .                          | 27        |
| 6.18     | Comparison with Disk Encryption . . . . .                    | 27        |
| 6.19     | Technical Complexity . . . . .                               | 27        |
| 6.20     | Software Engineering Complexity . . . . .                    | 28        |
| 6.21     | Major Project: Implementation of Enhanced Features . . . . . | 28        |
| 6.21.1   | Hardware-Backed Key Protection . . . . .                     | 28        |
| 6.21.2   | Encrypted Cloud Vault Synchronization . . . . .              | 28        |
| 6.21.3   | Detailed Access Audit Logs . . . . .                         | 29        |
| 6.21.4   | Enterprise-Level Deployment Support . . . . .                | 29        |
| 6.21.5   | Integration Architecture . . . . .                           | 29        |
| 6.22     | Academic Value . . . . .                                     | 30        |
| <b>7</b> | <b>Conclusions and Limitations</b>                           | <b>31</b> |
| 7.1      | Conclusions . . . . .                                        | 31        |
| 7.2      | Limitations . . . . .                                        | 32        |
| 7.2.1    | Runtime Trust Boundary . . . . .                             | 32        |
| 7.2.2    | Memory Hygiene . . . . .                                     | 32        |
| 7.2.3    | No Protection for Exported Files . . . . .                   | 32        |
| 7.2.4    | No Mounted Filesystem . . . . .                              | 32        |
| 7.2.5    | Hardware Key Protection Availability . . . . .               | 33        |

|       |                                       |           |
|-------|---------------------------------------|-----------|
| 7.2.6 | Cloud Synchronization Trust . . . . . | 33        |
|       | <b>References</b>                     | <b>34</b> |
|       | <b>Dissemination</b>                  | <b>36</b> |

# Chapter 1

## Introduction

### 1.1 Background

Digital information increasingly includes confidential documents, credentials, scanned certificates, personal records, and private archives. Such data often exists in heterogeneous storage environments, including laptops, removable drives, and cloud-backed folders. Conventional protection mechanisms such as full-disk encryption, operating system permissions, and ad hoc file compression do not always provide a simple, portable, and user-controlled way to protect a selected set of files [1]. AegisVault-J addresses this gap by offering a dedicated encrypted container with explicit user interaction boundaries.

### 1.2 Purpose of the Project

The purpose of AegisVault-J is to provide a digital safe for files. Rather than encrypting the entire disk or exposing a mounted filesystem, it stores user-selected content inside a single encrypted vault file. The design prioritizes clarity of operation, consistent security semantics, and portability across platforms. The user must consciously import files into the vault and export them back to the host system, ensuring that protected and unprotected states are never ambiguous.

## 1.3 Scope of the Work

The project scope includes vault creation, vault opening, importing and exporting files and folders, hierarchical vault organization, password change, auto-locking, backup support, password strength validation, and an audited cryptographic implementation. In the major project phase, the scope has been extended to include hardware-backed key protection using OS-level keystores, encrypted cloud vault synchronization via local folders and WebDAV, detailed access audit logging with a searchable viewer, and enterprise-level deployment support with cascading policy configuration and multi-vault management. The system protects the vault at rest and in transit as a file, but it does not protect exported files, malware-compromised hosts, or unlocked runtime memory from all threats.

## 1.4 Project Objectives

The major objectives are:

1. To create a portable encrypted container for selected files.
2. To implement authenticated encryption using AES-256-GCM.
3. To derive a password-based encryption key using Argon2id.
4. To maintain a secure separation between vault data and exported data.
5. To implement a cross-platform Java application with a user-friendly interface.
6. To validate the implementation using unit tests and security review.
7. To integrate hardware-backed key protection via OS-level keystores for master key wrapping.
8. To implement encrypted cloud vault synchronization supporting local folder and WebDAV providers.
9. To provide detailed access audit logging with a searchable, exportable audit trail.
10. To support enterprise-level deployment with cascading policy configuration, multi-vault management, and deployment reporting.

## **1.5 Problem Context**

Users who need to secure a small or medium set of sensitive files encounter several practical problems. Full-disk encryption is generally effective but is system-wide and may be complex to configure or explain. Manual file encryption is often tedious and prone to human error. Cloud vault services shift trust to third parties, introducing policy, availability, and privacy concerns. There is therefore a need for a simple, portable, and self-managed encrypted storage mechanism.

## **1.6 Motivation**

The motivation behind AegisVault-J is to make file protection operationally clear. Security should not depend on hidden background behavior or assumptions about implicit protection. By making import, encryption, locking, and export explicit steps, the system reduces user confusion and supports better security decisions.

## **1.7 Design Philosophy**

The project follows a security-first philosophy with honest threat modeling [2]. This means that the system is designed to do one task well: securely store selected data in an encrypted container. It intentionally avoids overclaiming protection against every possible adversary. This approach is academically valuable because it reflects realistic boundaries found in secure system design.

## Chapter 2

# Literature and Conceptual Context

### 2.1 Introduction

This chapter reviews existing approaches to file and data protection, placing AegisVault-J within the broader landscape of encryption tools and container systems. The review covers full-disk encryption, archive-based encryption, cloud vaults, and application-level container systems.

### 2.2 Full-Disk Encryption

Full-disk encryption protects an entire storage volume and is typically integrated into the operating system or boot chain [3]. It is useful when the objective is to protect data at rest for the whole device. However, it is broader than the requirements of a selective file vault. AegisVault-J differs by limiting protection to chosen files and by maintaining a visible import/export workflow.

### 2.3 Archive-Based Encryption

Encrypted archives and password-protected compressed files can protect groups of files, but they are often more suited to static bundles than to an interactive vault with folder hierarchy, backup, and file-management operations. AegisVault-J extends the archive idea into a controlled container with structured internal representation.

## **2.4 Cloud Vaults**

Cloud-based vaults offer convenience and synchronization, but they require trust in a third party and depend on network availability and service policies. AegisVault-J deliberately avoids that trust model by keeping the vault local and portable.

## **2.5 Application-Level Container Systems**

Application-level encrypted containers occupy a middle ground between simple archives and full filesystem encryption. They are suitable when the user wants secure file storage without kernel-level integration. AegisVault-J belongs to this category.

## Chapter 3

# System Model and Architecture

### 3.1 Introduction

This chapter presents the system model, architectural design, and module decomposition of AegisVault-J. It covers the frozen operational workflow, the layered architecture, individual layer responsibilities, and the module-level decomposition of the system.

### 3.2 Frozen Operational Workflow

The system uses a fixed Import → Encrypt → Lock → Export workflow. Files are imported into the vault as copies, not moved. They are then encrypted inside the container. The vault is locked when not in use. If the user needs a file outside the vault, it must be exported. Exported files are not protected by the vault after export.

### 3.3 Security Consequences of the Model

This workflow creates a clear line between protected and unprotected data. It helps users understand that the vault protects files only while they remain inside it. It also prevents a common misconception that an encryption tool automatically secures copies placed elsewhere on the host system. The exported file is ordinary file data and must be protected separately if needed.

## Operational Workflow of AejiVault-J

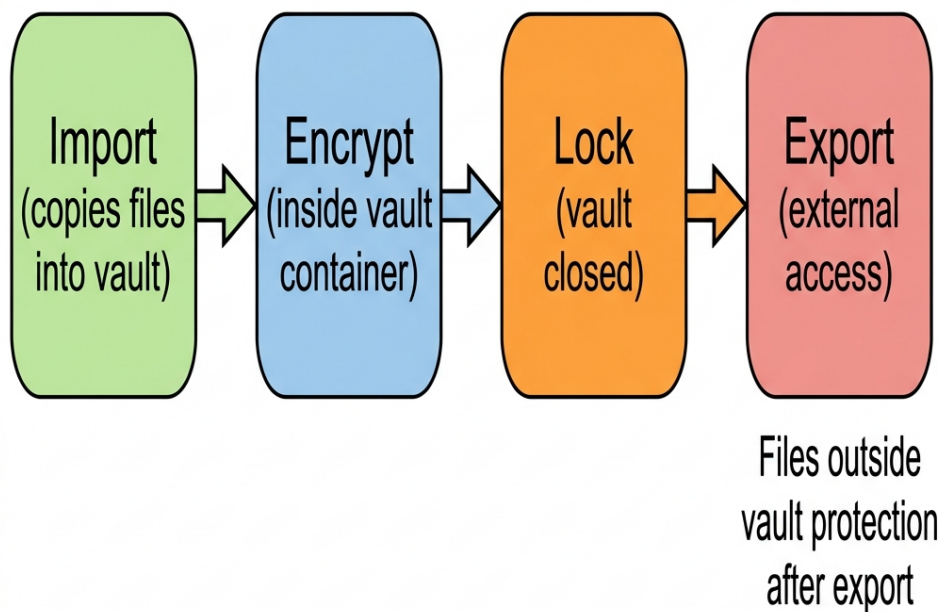


Figure 3.1: Operational workflow of AegisVault-J. The vault enforces a deliberate sequence of actions rather than hidden background encryption.

### 3.4 What the System Is

AegisVault-J is a digital safe for files, a portable encrypted container with the .avj extension, a Java-based cross-platform application, and a logically structured virtual file system.

### 3.5 What the System Is Not

The system is not disk encryption, not a mounted filesystem, not real-time file encryption, not protection for exported files, and not malware-resistant in the sense that a compromised host remains compromised. These exclusions are important because they prevent misleading security assumptions.

### 3.6 Layered Architecture

The architecture of the system is organized into five layers: JavaFX UI layer, service layer, virtual file system layer, encryption engine layer, and container file layer. Each layer has a narrow responsibility and communicates with adjacent layers through explicit interfaces. The layered architecture is shown in Figure 3.2.

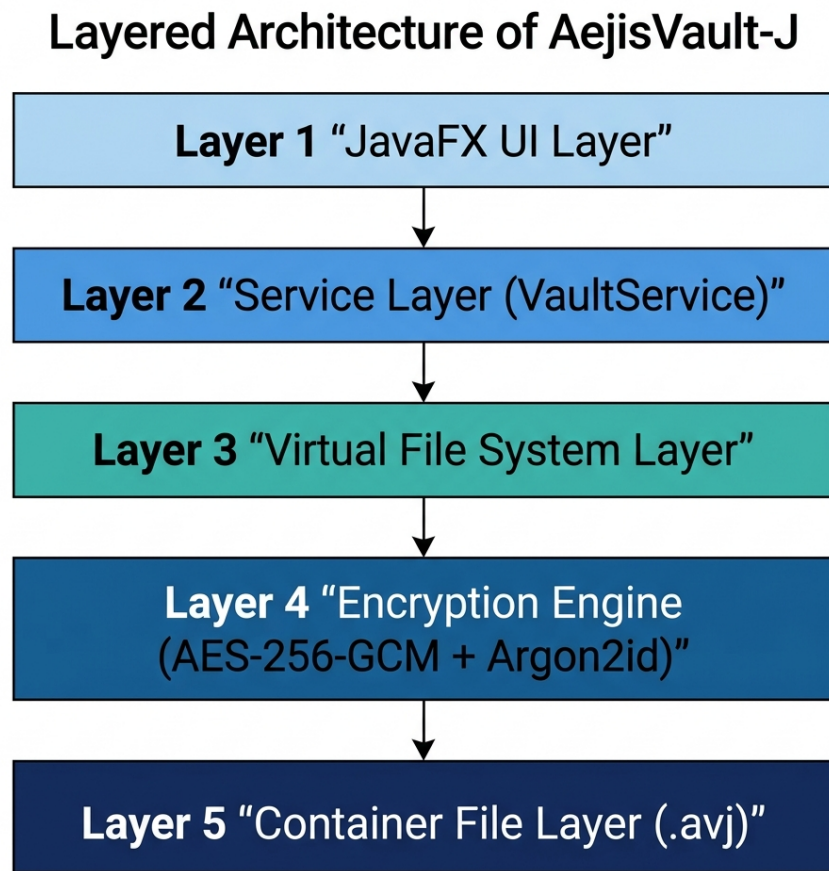


Figure 3.2: Layered architecture of AegisVault-J. The design separates presentation, business logic, logical file organization, cryptography, and persistent storage.

### 3.7 JavaFX UI Layer

The user interface is implemented with JavaFX . Its role is to present vault actions, accept credentials, show folder hierarchy, display security warnings, and guide the user through import and export flows. The UI layer must not perform cryptographic operations directly;

instead, it delegates to the service layer.

### 3.8 Service Layer

The `VaultService` coordinates application behavior. It validates user actions, enforces state transitions, and invokes file and cryptography operations in the proper order. Keeping these tasks in a service layer helps reduce coupling between UI elements and security-sensitive operations.

### 3.9 Virtual File System Layer

The virtual file system models the internal structure of the vault. It is logical rather than mounted at the operating system level. This design choice avoids kernel integration and reduces implementation complexity while still supporting folders and nested content.

### 3.10 Encryption Engine Layer

The encryption engine handles authenticated encryption, key generation, key derivation, key wrapping, and secure deletion attempts. It is the most security-sensitive layer and therefore requires careful testing and review.

### 3.11 Container File Layer

The `.avj` file is the persistent vault container. It stores encrypted metadata, encrypted content blobs, and cryptographic parameters required for opening the vault. Since the container is a single file, it is portable and can be transferred or backed up easily.

### 3.12 Module Design

The system is decomposed into four modules, each with a distinct responsibility. The module design is shown in Figure 3.3.

## Module Design of AejisVault-J

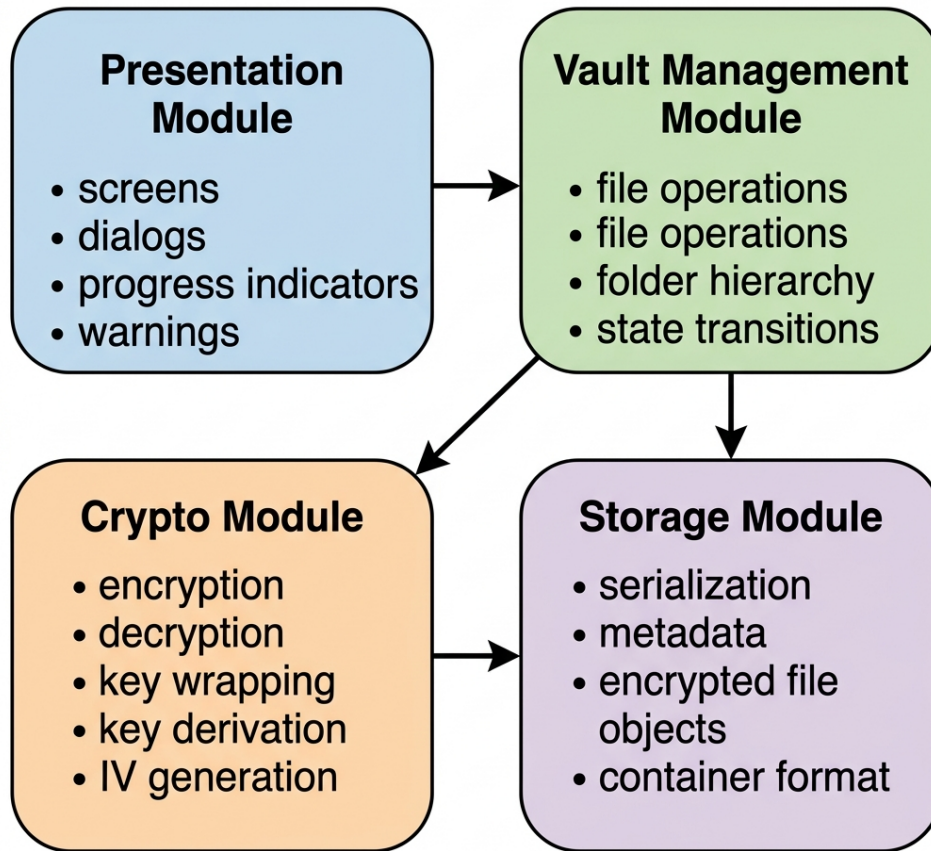


Figure 3.3: Module design of AejisVault-J showing the four core modules and their interactions.

### 3.12.1 Presentation Module

The presentation module is responsible for screens, dialog boxes, progress indicators, warning messages, and user interaction events. It must remain free from direct cryptographic implementation details.

### 3.12.2 Vault Management Module

The vault management module coordinates file operations, folder hierarchy updates, and state transitions between open, locked, and exported states.

### **3.12.3 Crypto Module**

The crypto module performs encryption, decryption, key wrapping, key derivation, and initialization vector generation. It must be isolated and well tested because errors here directly affect security.

### **3.12.4 Storage Module**

The storage module serializes metadata, encrypted file objects, and integrity-protected headers into the container file format. It also reconstructs them when opening the vault.

## **3.13 Enhanced Modules (Major Project)**

In the major project phase, four additional modules were integrated into the existing architecture, each adding significant enterprise-grade capability while preserving the original layered design.

### **3.13.1 Audit Module**

The audit module (`com.aegisvault.audit`) records every security-relevant operation performed within the vault. It tracks events such as vault creation, vault opening and closing, file imports, file deletions, file renames, password changes, and key protection mode changes. Each audit event is timestamped and stored within the encrypted vault itself, ensuring that the audit trail is protected by the same cryptographic guarantees as the vault data. A dedicated `AuditLogViewer` UI provides searchable, filterable, and exportable access to the audit trail.

### **3.13.2 Hardware Key Protection Module**

The hardware key protection module (`com.aegisvault.crypto`) enables the use of OS-level keystores for wrapping and unwrapping the Master Vault Key. On Windows, this leverages the `Windows-ROOT` keystore; on macOS, the `KeychainStore`; and on Linux, `PKCS#11` when available. The module supports three modes: software-only

(default), OS keystore, and OS keystore with software fallback. This provides defense-in-depth by binding vault keys to the platform's trusted hardware or secure enclave.

### **3.13.3 Cloud Synchronization Module**

The cloud synchronization module (`com.aegisvault.sync`) enables encrypted vault files to be synchronized to remote storage. Two providers are implemented: local folder synchronization (for network shares and external drives) and WebDAV synchronization (for cloud-based storage). The `SyncEngine` handles upload, download, and conflict detection. Synchronization can be triggered manually, on vault close, or on a scheduled interval. Enterprise policies can disable synchronization entirely.

### **3.13.4 Enterprise Deployment Module**

The enterprise deployment module (`com.aegisvault.enterprise`) supports organizational deployment of AegisVault-J. It includes a cascading configuration loader that reads policies from application defaults, user home directory, system-wide configuration, and Windows registry (Group Policy). Enterprise administrators can enforce minimum password lengths, maximum auto-lock timeouts, allowed ciphers, and synchronization permissions. The module also includes multi-vault management with health checks, silent installation configuration, and deployment report generation for IT administrators.

## Chapter 4

# Cryptographic Design

### 4.1 Introduction

This chapter presents the cryptographic design of AegisVault-J, covering the core key concepts, key derivation flow, encryption algorithms, security properties, entropy sources, and the supplementary mouse randomness feature.

### 4.2 Core Key Concepts

The design uses two key concepts. The first is the Master Vault Key (MVK) , a random 256-bit symmetric key that encrypts all vault data. The second is the Key Encryption Key (KEK) , a password-derived key created from the user's password using Argon2id. The Key Encryption Key (KEK) is used only to encrypt or decrypt the Master Vault Key (MVK) [4,5] and is never stored in plaintext.

### 4.3 Key Derivation and Unlock Flow

The unlock process is intentionally indirect. The password is never compared with a stored password hash for vault access. Instead, the system derives a Key Encryption Key (KEK) from the password and uses that key to decrypt the Master Vault Key (MVK). If the decryption succeeds and the authentication tag verifies correctly, the password is accepted. If it fails, the password is rejected. The key derivation and unlock flow is

shown in Figure 4.1.

## Key Derivation and Unlock Flow

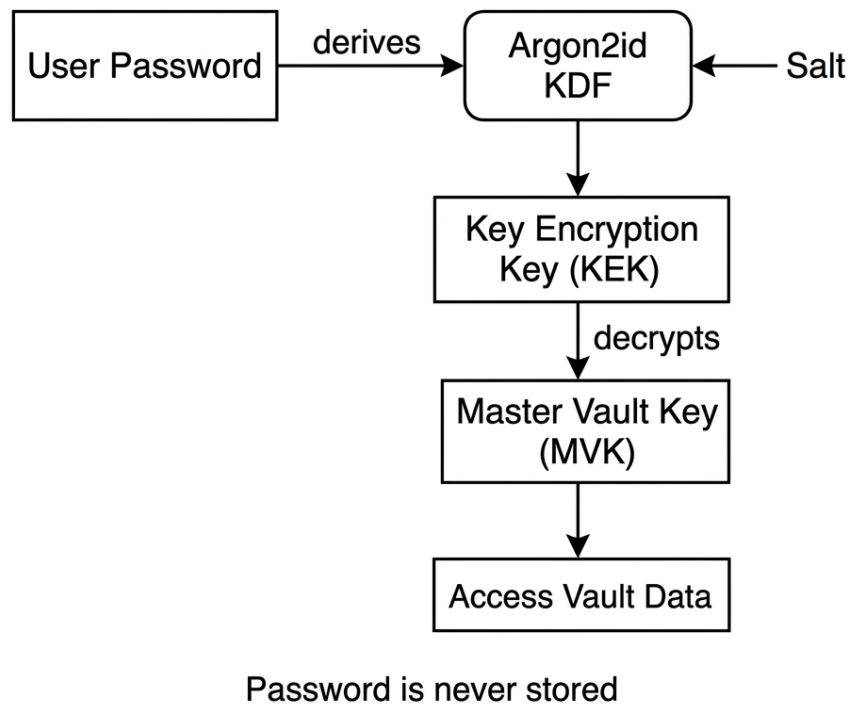


Figure 4.1: Password-to-vault access flow. Authentication succeeds through correct cryptographic decryption rather than direct password comparison.

## 4.4 AES-256-GCM

AES-GCM [6, 7] is used because it provides confidentiality and integrity in a single mode of operation. AES-256 offers a strong key size, while GCM provides authenticated encryption with associated data [8]. In the vault context, this means tampering with ciphertext is detected, and a modified encrypted blob will fail validation during decryption.

## 4.5 Argon2id

Argon2id [9] is a password-based key derivation function designed to be resistant to GPU and ASIC acceleration while also defending against tradeoffs between memory and computation. The project uses a memory cost of 64 MB, which raises the cost of brute-force attempts. The exact choice reflects a balance between security and usability on typical student hardware.

## 4.6 Security Properties

The cryptographic design provides several important properties:

- the password is never stored in plaintext,
- there is no direct password comparison table,
- vault access is bound to successful authenticated decryption,
- encrypted container contents remain protected while stored,
- tampering with stored ciphertext is detected during decryption.

## 4.7 Entropy Sources

The system uses `SecureRandom` for cryptographic randomness. Optional mouse entropy can be mixed in as a supplementary source, but it is never the primary entropy source. Security is therefore not dependent on human motion or user behavior.

## 4.8 Mouse Randomness Feature

### 4.8.1 Design Rationale

Some implementations use user interaction data as an auxiliary entropy source. In AegisVault-J, mouse movements are optionally collected and converted to byte entropy. This design may improve entropy diversity, but it is carefully positioned as a supplementary feature only.

### **4.8.2 Processing Model**

The application records mouse coordinates and timestamps, transforms them into bytes, and mixes them into the random generator through reseeding. Because cryptographic security must not depend on user behavior, this feature is not part of the trust base.

### **4.8.3 Security Interpretation**

The feature must not be treated as primary entropy. If the mouse data were unavailable or predictable, the vault security would still rely on proper cryptographic primitives and a strong password. This distinction is important for honest security analysis.

## **Chapter 5**

# **Security Features and Threat Model**

### **5.1 Introduction**

This chapter describes the security features implemented in AegisVault-J and presents the honest threat model that defines what the system protects and what falls outside its protection boundary. The security model is illustrated in Figure 5.1.

### **5.2 Authenticated Encryption**

Authenticated encryption ensures that confidentiality and integrity are both enforced. For a vault system, integrity is as important as secrecy, because corrupted ciphertext should not be silently accepted.

### **5.3 Auto-Lock**

The auto-lock feature reduces exposure when the user leaves the application idle. This does not eliminate all risk, but it reduces the duration in which sensitive data remains open in memory and in the interface state.

## Security Model of AegisVault-J

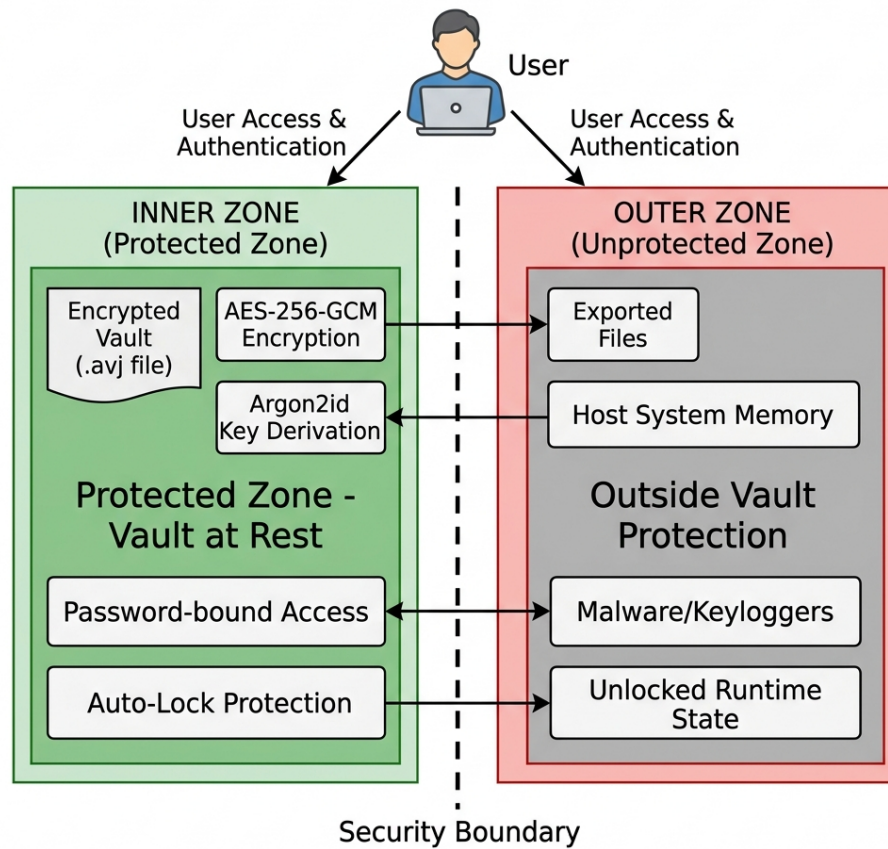


Figure 5.1: Security model of AegisVault-J showing the boundary between protected vault data and unprotected external data.

## 5.4 Key Wiping

The application attempts to clear in-memory copies of sensitive key material after use. This is beneficial but should be understood as best-effort mitigation rather than a guarantee, because runtime environments may retain intermediate copies in internal structures.

## 5.5 Export Warnings

Since exported files are outside vault protection, the application warns users when they intentionally remove content from secure storage. This helps preserve the clarity of the security model.

## 5.6 Password Strength Checking

Password strength feedback is included to guide users toward better vault passwords [10, 11]. Strong passwords increase the work factor of brute-force attacks, especially when combined with Argon2id.

## 5.7 Hardware-Backed Key Protection

The major project introduces hardware-backed key protection as an additional layer of defense. When enabled, the Master Vault Key is wrapped using the operating system's native keystore (Windows-ROOT, macOS KeychainStore, or Linux PKCS#11). This means that even if an attacker obtains the encrypted vault file, they cannot extract the master key without access to the specific machine's keystore. The system supports three modes: software-only (default), OS keystore, and OS keystore with software fallback. Mode changes are recorded in the audit log for accountability.

## 5.8 Access Audit Logging

The audit logging system records every security-relevant operation: vault creation, opening, closing, file imports, deletions, renames, password changes, and key protection mode changes. Each event is timestamped and stored within the encrypted vault itself, ensuring that the audit trail receives the same cryptographic protection as vault data. The audit log supports search, filtering by event type, and export to CSV for external analysis. This feature is essential for enterprise compliance and for detecting unauthorized access patterns.

## 5.9 Protected Assets

The system protects the vault while it is stored as an encrypted file and while it is transferred between locations. It also raises the cost of offline brute-force attempts by using Argon2id and AES-GCM correctly.

## 5.10 Out of Scope

The system does not protect exported files, malware-infected endpoints, keyloggers, memory forensics, or the unlocked runtime state. If a hostile program runs on the host, it may still observe user activity or memory contents.

## 5.11 Boundary of Responsibility

The boundary of responsibility is a central academic point in the project. Good security software must state clearly what it protects and what it does not protect. AegisVault-J is designed to be honest in this regard.

Table 5.1: Security model summary of AegisVault-J

| Category                         | Status                                                     |
|----------------------------------|------------------------------------------------------------|
| Vault at rest                    | Protected by AES-256-GCM and password-derived key wrapping |
| Vault transfer/storage as a file | Protected as an encrypted container                        |
| Brute-force resistance           | Improved by Argon2id and strong password guidance          |
| Exported files                   | Not protected by the vault                                 |
| Malware/keyloggers               | Not protected                                              |
| Memory forensics                 | Not guaranteed to be protected                             |
| Unlocked vault state             | Exposed to the current runtime trust boundary              |

## **Chapter 6**

# **Implementation, Testing, and Results**

### **6.1 Introduction**

This chapter covers the implementation stack, functional features, testing strategy, comparison with other systems, and complexity analysis of AegisVault-J.

### **6.2 Language and Runtime**

The application is implemented in Java [12]. Java provides portability, rich library support, and strong ecosystem compatibility with desktop application development.

### **6.3 User Interface**

JavaFX [13] is used for the desktop UI. It supports modern interface rendering, event handling, and structured layout without requiring platform-specific UI code.

### **6.4 Application UI Flow**

The application presents a structured user interface that guides the user through vault operations. The UI flow consists of three primary screens: vault creation/login, vault file browser, and import/export dialogs.

### 6.4.1 Vault Creation and Login Screen

The initial screen allows the user to either create a new vault or open an existing one. During vault creation, the user provides a password which is validated using the built-in password strength checker. The password strength indicator provides real-time feedback to help users choose strong vault passwords. Once a vault is created or opened, the user is directed to the main vault file browser. The vault creation screen is shown in Figure 6.1.

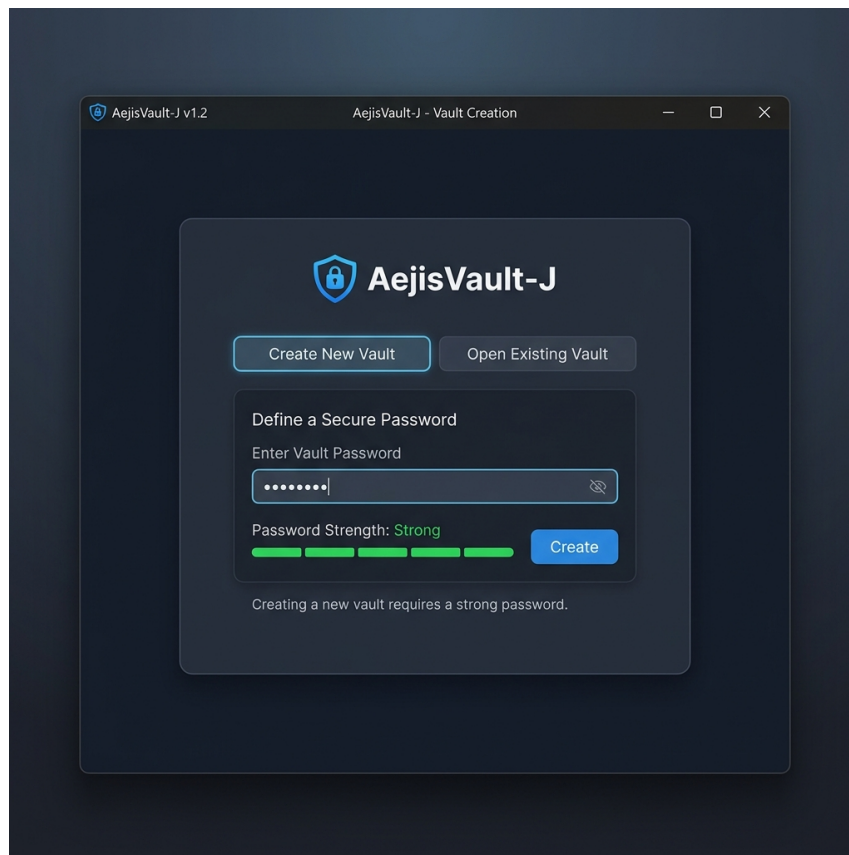


Figure 6.1: Vault creation and login screen of AegisVault-J. Users can create a new vault or open an existing one with password authentication.

### 6.4.2 Main Vault File Browser

After successful authentication, the user is presented with the main vault file browser interface. The left panel displays the folder hierarchy within the vault, and the right panel shows the contents of the selected folder. The toolbar provides access to core operations: importing files, exporting files, creating new folders, deleting items, locking the vault, and accessing settings. The vault file browser is shown in Figure 6.2.

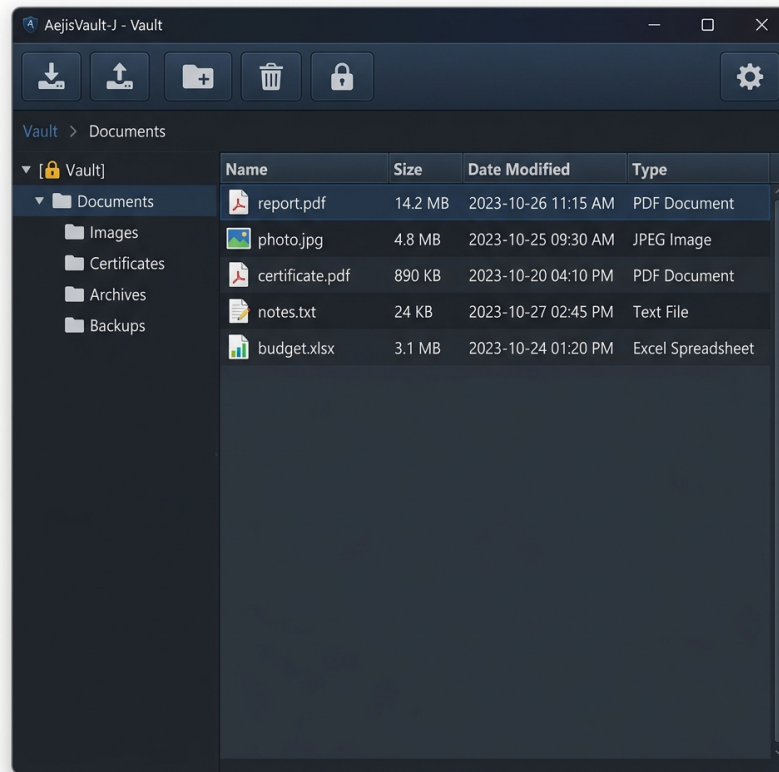


Figure 6.2: Main vault file browser of AegisVault-J. The interface provides a familiar file manager layout with vault-specific operations in the toolbar.

### 6.4.3 Import and Export Operations

The import dialog allows users to select files from the host filesystem and encrypt them into the vault. A progress indicator shows the encryption status during the import process. The export dialog similarly allows users to select vault items and decrypt them to the host filesystem, with an explicit warning that exported files will no longer be protected by the vault. The import dialog is shown in Figure 6.3.

## 6.5 Cryptographic Libraries

The project uses `javax.crypto` alongside BouncyCastle [14] for cryptographic operations. This combination allows access to standard Java cryptography APIs and supplemental algorithm support where needed.

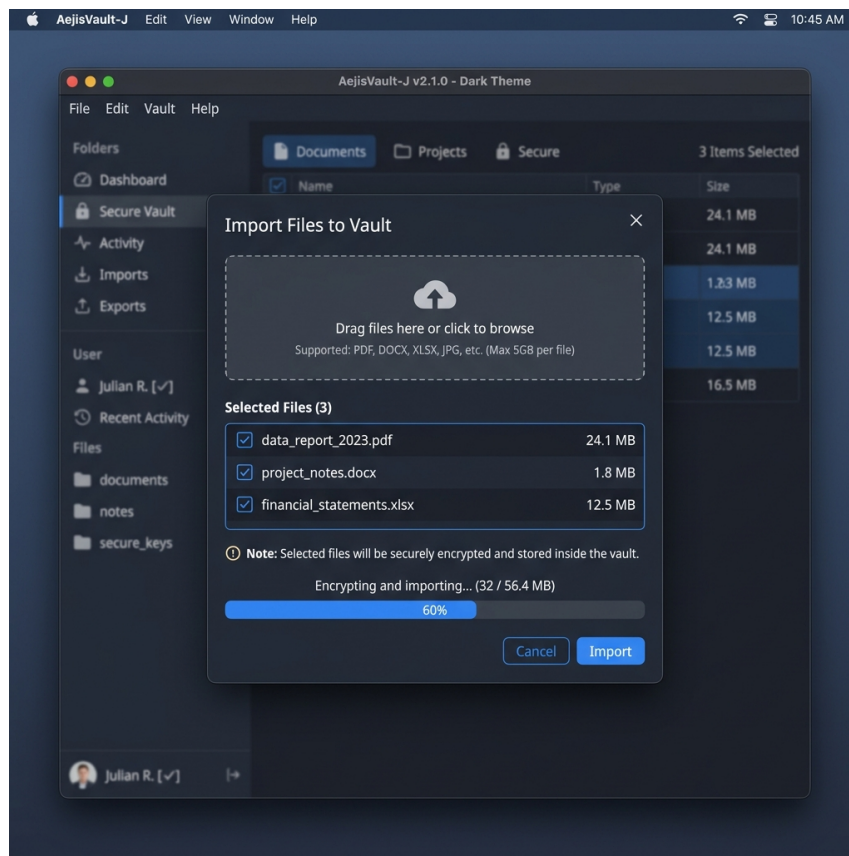


Figure 6.3: Import/Export dialog of AegisVault-J. The interface shows file selection, encryption progress, and security warnings for export operations.

## 6.6 Build and Packaging

Gradle manages the build process, dependency resolution, and packaging tasks. The project can be targeted for native distribution formats such as Windows `.exe`, Linux AppImage, and macOS `.app`.

## 6.7 Vault Lifecycle

The system supports vault creation and opening. During creation, the vault structure is initialized, and the encrypted key material is stored. During opening, the password-based key derivation process is performed and the vault contents are decrypted if authentication succeeds.

## **6.8 Import and Export Operations**

Import operations copy files or folders into the vault. Export operations copy selected items out of the vault to a chosen host path. Since export means leaving the protected environment, the system marks this as an explicit trust boundary.

## **6.9 Folder Hierarchy Management**

The vault preserves an internal folder tree. This helps users organize content in a way that resembles a normal file manager while still keeping the data inside the encrypted container.

## **6.10 Password Change**

The password change feature rewraps the Master Vault Key (MVK) with a new Key Encryption Key (KEK) derived from the new password. This design avoids re-encrypting every file in the vault and reduces the risk and cost of password updates.

## **6.11 Backup Utility**

Backup support helps users preserve encrypted vault copies. Since the vault file is self-contained, backups can be made at the container level rather than by copying many individual files.

## **6.12 Password Strength Checker**

The password strength checker offers immediate feedback on likely weakness patterns such as short length or common structure. It does not replace cryptographic protection, but it helps improve user behavior.

## 6.13 Unit Testing Strategy

The project includes more than 80 unit tests spanning cryptography, virtual file system logic, container serialization, and service-layer behavior. Comprehensive testing is important because security code frequently fails in edge cases.

## 6.14 Coverage Areas

The tests cover:

- encryption and decryption correctness,
- wrong-password handling,
- container read/write behavior,
- folder and file tree logic,
- service-layer state transitions,
- password-change operations,
- auto-lock behavior and warnings.

## 6.15 Security Audit Summary

A security audit was completed and did not identify critical vulnerabilities in the design as implemented. This does not mean the system is invulnerable; rather, it indicates that no severe issues were found within the examined scope and assumptions.

## 6.16 Observed JVM Limitations

The audit identified some practical limitations inherited from the runtime environment:

- Java password input components may internally use `String` representations.
- Memory zeroing cannot be absolutely guaranteed for all intermediate copies.

- Encoding and framework operations may create temporary data copies.

These observations are important because they prevent overstatement of memory hygiene guarantees.

## 6.17 Comparison with VeraCrypt

VeraCrypt [3] is a mature disk and volume encryption tool with kernel-level integration and broad system protection. AegisVault-J differs in that it is Java-only, application-level, and centered on an explicit import/export model. This makes it simpler to understand for controlled workflows, though it does not offer the same class of system-wide protection.

## 6.18 Comparison with Disk Encryption

Disk encryption protects the entire system volume. AegisVault-J protects selected files only. This narrower scope gives the user more flexibility and portability, but it also means that protection is limited to the vault boundary.

Table 6.1: Comparison of AegisVault-J with common alternatives

| Feature           | AegisVault-J       | VeraCrypt          | Disk Encryption    |
|-------------------|--------------------|--------------------|--------------------|
| Protection scope  | Selected files     | Volumes/containers | Entire disk/system |
| User model        | Import/export      | Mounted/container  | Transparent        |
| Platform          | Java application   | Native integration | OS-level           |
| Portability       | High               | High to moderate   | Low to moderate    |
| Granularity       | File-level control | Volume-level       | Whole-device       |
| Kernel dependency | None               | Low-level          | High               |

## 6.19 Technical Complexity

The project combines several advanced topics: cryptography, secure key management, logical file system abstraction, desktop UI design, and memory safety considerations. Even though the implementation is application-level, the conceptual design is nontrivial because each layer must preserve security boundaries.

## 6.20 Software Engineering Complexity

The software engineering challenge lies in coordinating usability and security. A system that is secure but hard to understand may be used incorrectly, while a system that is easy to use but insecure undermines its purpose. The explicit workflow chosen here attempts to balance these concerns.

## 6.21 Major Project: Implementation of Enhanced Features

The major project phase extends AegisVault-J with four enterprise-grade features. Each feature is described below.

### 6.21.1 Hardware-Backed Key Protection

The `HardwareKeyProtection` class provides OS-level keystore integration for wrapping and unwrapping the Master Vault Key [15, 16]. On Windows, the `Windows-ROOT` keystore is used; on macOS, the `KeychainStore`; and on Linux, `PKCS#11` when available. The `KeyProtectionConfig` class persists the user's selected mode (software-only, OS keystore, or OS keystore with fallback) to the user preferences [17]. The implementation uses only `java.security.KeyStore` APIs to avoid native code dependencies.

### 6.21.2 Encrypted Cloud Vault Synchronization

The synchronization subsystem consists of a provider interface (`CloudSyncProvider`), two concrete providers (`LocalFolderSyncProvider` and `WebDavSyncProvider`), a synchronization engine (`SyncEngine`), and a configuration model (`SyncConfig`). The local folder provider performs atomic file copy with rename-on-success to prevent partial uploads. The WebDAV provider uses `java.net.http.HttpClient` for PUT and GET operations with Basic authentication [18, 19]. The engine supports manual sync, sync-on-close, and interval-based scheduling. Conflict detection uses file modification timestamps.

### 6.21.3 Detailed Access Audit Logs

The audit subsystem records every security-relevant vault operation [20]. The `AuditLog` class maintains an in-memory list of `AuditEvent` records and persists them as a JSON log within the encrypted vault. Events include vault creation, opening, closing, file imports, deletions, renames, password changes, and key protection mode changes. The `AuditLogViewer` is a JavaFX dialog that provides a searchable table view with type-based filtering and CSV export capability. The `VaultService` is instrumented to call `recordAudit()` at each operation boundary.

### 6.21.4 Enterprise-Level Deployment Support

The enterprise module includes five classes. `EnterpriseConfig` models organizational policies such as minimum password length, maximum auto-lock timeout, allowed ciphers, sync permissions, and policy-locked settings [21]. `EnterpriseConfigLoader` implements cascading configuration from four sources: application defaults, user home directory, system-wide configuration file, and Windows registry (for Group Policy integration). `MultiVaultManager` maintains a persistent registry of vault files with health checks and labels. `DeploymentReport` generates JSON reports with system information, configuration status, and vault inventory. `EnterpriseSettingsDialog` provides a tabbed JavaFX interface for viewing policies, managing vault inventory, and exporting deployment reports [22].

### 6.21.5 Integration Architecture

All four features integrate through the `VaultService` class, which serves as the central coordination point. The service holds references to the audit log, sync engine, sync config, key protection config, and enterprise config. The `MainApplication` class initializes all configurations at startup, and the `MainController` provides UI access to each feature through menu items and keyboard shortcuts.

## **6.22 Academic Value**

From an academic perspective, the project demonstrates secure design tradeoffs, threat modeling discipline, and the use of strong modern cryptographic primitives. The major project phase significantly extends the system with enterprise-grade features including hardware key protection, cloud synchronization, audit logging, and policy-based deployment. It is suitable for a B.Tech Major Project because it combines practical security engineering with enterprise software architecture and demonstrates the ability to evolve a system from a personal tool to an organizational platform.

## Chapter 7

# Conclusions and Limitations

### 7.1 Conclusions

AegisVault-J is a secure encrypted container system that demonstrates practical application of authenticated encryption, password-based key derivation, controlled file management, and explicit security boundaries. Its main contribution is not merely technical encryption, but the clear operational model that distinguishes protected vault data from unprotected exported files.

In the major project phase, the system has been significantly extended with four enterprise-grade features that were previously identified as future enhancements:

- **Hardware-Backed Key Protection:** OS-level keystore integration (Windows-ROOT, macOS KeychainStore, Linux PKCS#11) enables the Master Vault Key to be wrapped using platform-specific hardware security, providing defense-in-depth beyond password-based protection.
- **Encrypted Cloud Vault Synchronization:** Local folder and WebDAV synchronization providers allow encrypted vault files to be backed up and synchronized across locations. Conflict detection, sync-on-close, and enterprise policy enforcement ensure data integrity.
- **Detailed Access Audit Logs:** Every security-relevant vault operation is recorded with timestamps and stored within the encrypted vault. A searchable, filterable, and exportable audit viewer enables compliance monitoring and forensic analysis.

- **Enterprise-Level Deployment Support:** Cascading policy configuration (defaults → user → system → registry), multi-vault management, silent installation support, and deployment reporting enable organizational deployment and IT administration.

These enhancements elevate AegisVault-J from a personal file vault to an enterprise-ready secure storage platform. The project is valuable as a B.Tech Major Project because it integrates cryptography, application design, secure storage concepts, enterprise software architecture, and honest threat modeling into a coherent software system.

The system does not claim absolute security. Instead, it provides a realistic and well-defined security tool with clear protections, explicit limitations, and an architecture suited to both personal and organizational data storage.

## 7.2 Limitations

### 7.2.1 Runtime Trust Boundary

Once the vault is open, the application depends on the security of the host runtime. If the host is compromised, the vault cannot fully defend itself. This is a standard limitation for user-space security tools.

### 7.2.2 Memory Hygiene

Best-effort wiping does not guarantee that all sensitive material disappears from memory. Garbage collection, JIT compilation, temporary buffers, and framework internals can all retain data longer than intended.

### 7.2.3 No Protection for Exported Files

Exported files are ordinary files and must be protected separately. This limitation is deliberate and consistent with the design model.

### 7.2.4 No Mounted Filesystem

The project does not use FUSE, Dokany, or a kernel-integrated mount layer. This keeps the application lightweight but means it behaves like a container manager rather than a

transparent filesystem.

### **7.2.5 Hardware Key Protection Availability**

The hardware-backed key protection depends on the availability of OS-level keystores, which may vary across platforms and configurations. On systems without supported keystores, the system falls back to software-only key protection.

### **7.2.6 Cloud Synchronization Trust**

While the vault file is encrypted before synchronization, the synchronization endpoints themselves (network shares, WebDAV servers) are trusted to store the encrypted data reliably. Network availability and endpoint security are outside the system's control.

## References

- [1] Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno. *Cryptography Engineering: Design Principles and Practical Applications*. Wiley, 1st edition, 2010.
- [2] Gary McGraw. Software security: Building security in. In *IEEE Security & Privacy*, volume 2, pages 80–83. IEEE, 2004.
- [3] Pascal Junod and Alain Karber. A security analysis of VeraCrypt: Taking a closer look at full disk encryption. *IACR Cryptology ePrint Archive*, 2020. Report 2020/1494.
- [4] National Institute of Standards and Technology. Recommendation for block cipher modes of operation: Methods for key wrapping. NIST Special Publication 800-38F, 2012. U.S. Department of Commerce.
- [5] Shai Halevi and Phillip Rogaway. A tweakable enciphering mode. In *Advances in Cryptology – CRYPTO 2003*, pages 482–499. Springer, 2003.
- [6] National Institute of Standards and Technology. Recommendation for block cipher modes of operation: Galois/Counter mode (GCM) and GMAC. NIST Special Publication 800-38D, 2007. U.S. Department of Commerce.
- [7] National Institute of Standards and Technology. Advanced encryption standard (AES). Federal Information Processing Standards Publication 197, 2001. U.S. Department of Commerce.
- [8] Phillip Rogaway. Authenticated-encryption with associated-data. *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, pages 98–107, 2002.
- [9] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: New generation of memory-hard functions for password hashing and other applications. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 292–302. IEEE, 2016.
- [10] Joseph Bonneau. The science of guessing: Analyzing an anonymized corpus of 70 million passwords. In *2012 IEEE Symposium on Security and Privacy*, pages 538–552. IEEE, 2012.
- [11] National Institute of Standards and Technology. Digital identity guidelines: Authentication and lifecycle management. NIST Special Publication 800-63B, 2017. U.S. Department of Commerce.

- [12] Scott Oaks. *Java Security*. O'Reilly Media, 2nd edition, 2001. Comprehensive guide to Java platform security architecture.
- [13] Oracle Corporation. JavaFX documentation. <https://openjfx.io/>, 2024. Rich client platform for Java desktop applications.
- [14] The Legion of the Bouncy Castle. The Legion of the Bouncy Castle – Java cryptography APIs. <https://www.bouncycastle.org/>, 2024. Open-source cryptographic API provider for Java.
- [15] National Institute of Standards and Technology. Security requirements for cryptographic modules. Federal Information Processing Standards Publication 140-3, 2019. U.S. Department of Commerce.
- [16] Oracle Corporation. Java platform, standard edition security developer's guide. <https://docs.oracle.com/en/java/javase/21/security/>, 2024. KeyStore, KeyManager, and TrustManager API documentation.
- [17] Will Arthur, David Challener, and Kenneth Goldman. A practical guide to TPM 2.0: Using the new Trusted Platform Module in the new age of security. *Apress*, 2015.
- [18] Lisa Dusseault. Http extensions for web distributed authoring and versioning (WebDAV). RFC 4918, IETF, 2007. Internet Engineering Task Force.
- [19] S. Subashini and V. Kavitha. A survey on security issues in service delivery models of cloud computing. volume 34, pages 1–11. Elsevier, 2011.
- [20] National Institute of Standards and Technology. Guide to computer security log management. NIST Special Publication 800-92, 2006. U.S. Department of Commerce.
- [21] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996.
- [22] National Institute of Standards and Technology. Recommendation for key management: Part 1 – general. NIST Special Publication 800-57 Part 1, Revision 5, 2020. U.S. Department of Commerce.

# Dissemination

## B.Tech Minor Project

1. **Ayshi Shannidhya Panda, Ashish Singh, Aves Ranjan, and Ashish Kumar**, “AegisVault-J: Secure Encrypted Container System”, B.Tech Minor Project, Department of Computer Science and Engineering, Silicon Institute of Technology, Sambalpur, Odisha, India, April 2026.

## Technologies and Tools Used

1. Java (OpenJDK) – Core programming language
2. JavaFX – Desktop UI framework
3. BouncyCastle – Cryptographic library (AES-256-GCM, Argon2id)
4. Gradle – Build and dependency management
5. JUnit – Unit testing framework (80+ tests)